
DevOps: development - operations

Michele Marchesi

marchesi@unica.it

**Corso di Ingegneria del Software
A.A. 2024/25**



DevOps

- ◆ DevOps è una metodologia di ingegneria del software che nasce dalla difficoltà di comunicazione tra chi scrive il software (development) e chi lo deve rilasciare e mantenere in produzione (operations)
- ◆ Il termine "DevOps" è stato coniato da Patrick Debois nel 2009
- ◆ DevOps integra e automatizza lo sviluppo software (Dev) e le operazioni IT (Ops) per migliorare e abbreviare il ciclo di vita dello sviluppo dei sistemi software
- ◆ L'obiettivo non è soltanto “rilasci più frequenti”, ma ridurre il tempo che separa una modifica del codice da un **uso affidabile in produzione**
- ◆ DevOps è l'applicazione della metodologia agile all'amministrazione dei sistemi *(Tom Limoncelli)*



Operations

Attività	Descrizione
Hosting	Preparazione di server, servizi, reti
Deploy	Installazione del software su ambienti di test e produzione
Monitoring & alerting	Controllo dello stato del sistema, dei log e delle metriche
Logging	Registrazione e analisi degli eventi del sistema
Security	Gestione di accessi, firewall, backup
Scaling	Adattamento a carico e crescita del servizio
Gestione incidenti	Diagnosi e risoluzione rapida di errori e anomalie
Business continuity	Ridondanza, failover, disaster recovery



Ruoli di Ops

Ruolo	Compito principale
System Admin	Amministrazione di sistemi e ambienti
Site Reliability Engineer (SRE)	Affidabilità e prestazioni di Ops
DevOps Engineer	Collabora con Dev, automatizza tutto il ciclo di delivery
Cloud Engineer	Gestione di infrastrutture cloud e servizi gestiti
Security Engineer	Garantisce la sicurezza dei sistemi

Nelle organizzazioni moderne questi ruoli tendono a collaborare strettamente con gli sviluppatori (il Team *Dev*), condividendo obiettivi di affidabilità e lead time.

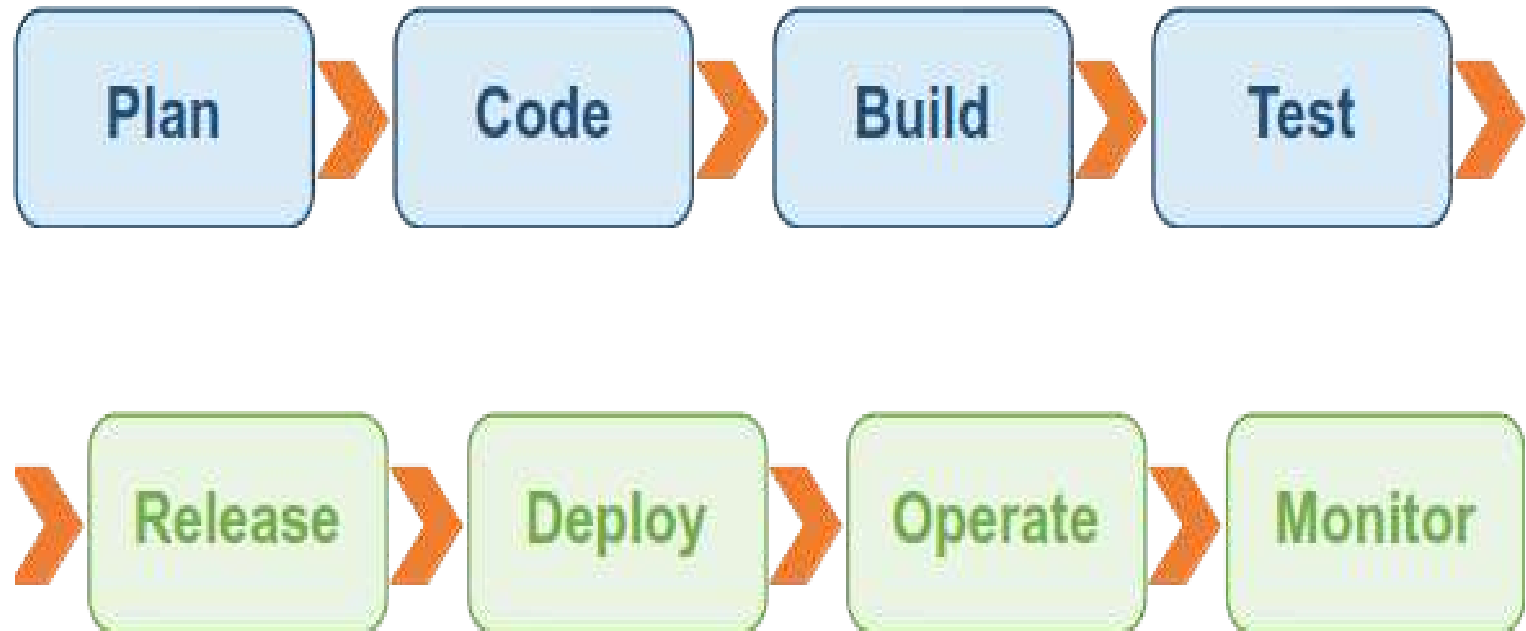


Ops

- ◆ In passato, il mondo Ops era spesso separato, lento, burocratico.
- ◆ Con DevOps, SRE e l'Infrastructure as Code, il mondo Ops oggi è:
 - automatizzato
 - integrato con lo sviluppo
 - basato su strumenti moderni (Terraform, Kubernetes, Docker, Ansible)
 - fondamentale per il Continuous Delivery

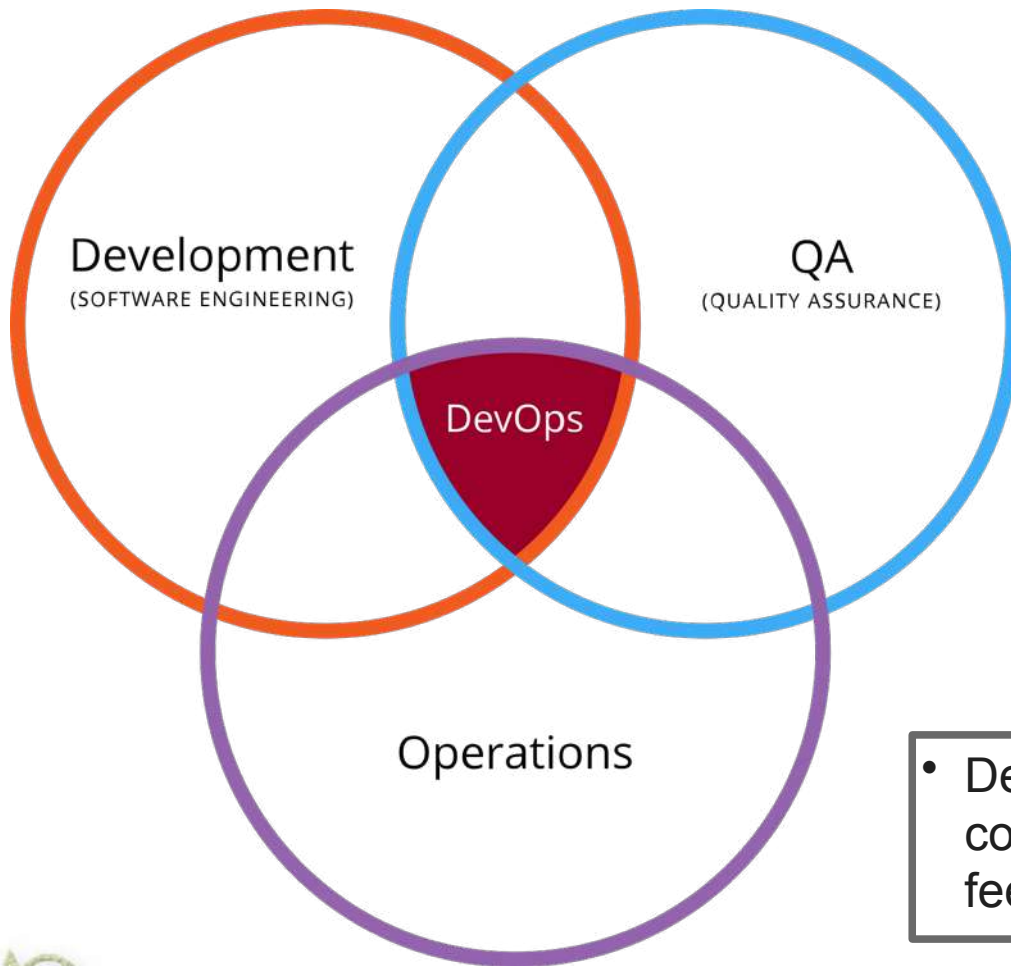


DevOps nel ciclo di vita



Il ciclo non termina con il rilascio: le informazioni raccolte in esercizio ritornano alla pianificazione e alle decisioni di sviluppo.

Intersezione di Dev, QA e Ops



- DevOps non elimina QA o Ops: coordina responsabilità, strumenti e feedback tra funzioni diverse.

Intersezione di Dev, QA e Ops

- ◆ **DEV – Sviluppo**
- ◆ **Chi:** programmatori, architetti software
- ◆ **Obiettivi:** scrivere codice di qualità, aggiungere funzionalità, risolvere bug
- ◆ **Attività:** coding, versionamento (Git), code review, test unitari
- ◆ **Strumenti:** Git, VS Code, Spring, Django, Docker
- ◆ **Obiettivo complessivo:** massimizzare la capacità di evolvere il prodotto senza introdurre difetti



Intersezione di Dev, QA e Ops

- ◆ **QA – Quality Assurance**
- ◆ **Chi:** tester, QA engineer, automation specialist
- ◆ **Obiettivi:** validare funzionalità, prevenire regressioni
- ◆ **Attività:** test manuali e automatici, bug tracking, verifica criteri di accettazione
- ◆ **Strumenti:** Selenium, pytest, Playwright, Postman, Jenkins, SonarQube
- ◆ **Obiettivo complessivo:** aumentare la confidenza nel rilascio, riducendo il costo degli errori di regressione scoperte tardi

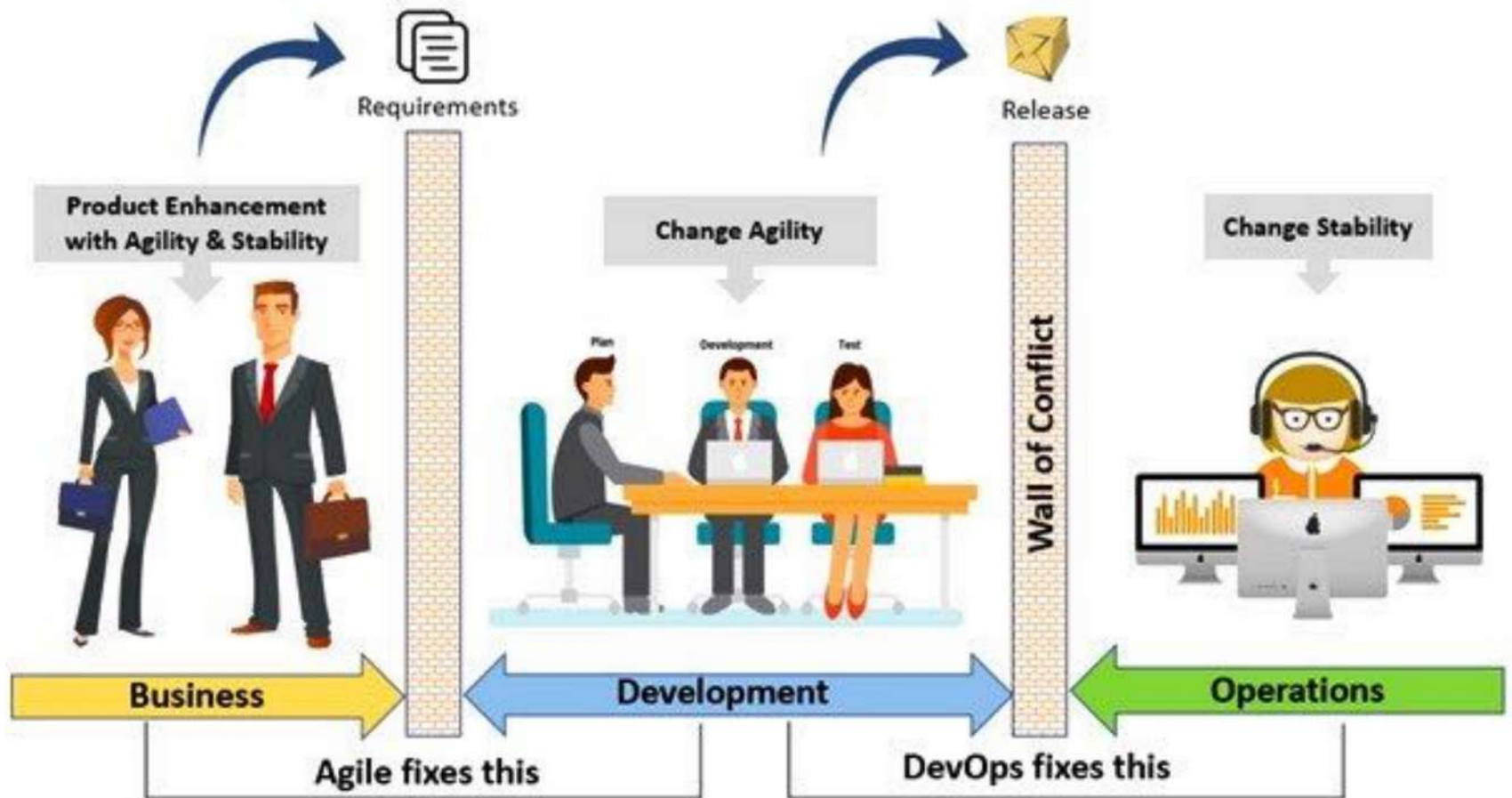


Intersezione di Dev, QA e Ops

- ◆ **OPS – Operations**
- ◆ **Chi:** sysadmin, DevOps/SRE, cloud engineer
- ◆ **Obiettivi:** stabilità, sicurezza, rilasci continui
- ◆ **Attività:** provisioning, monitoring, CI/CD, gestione ambienti, backup, capacity planning, incident management, ripristino
- ◆ **Strumenti:** Kubernetes, Terraform, Prometheus, piattaforme Cloud
- ◆ **Obiettivo complessivo:** mantenere il servizio disponibile, osservabile e riproducibile nel tempo



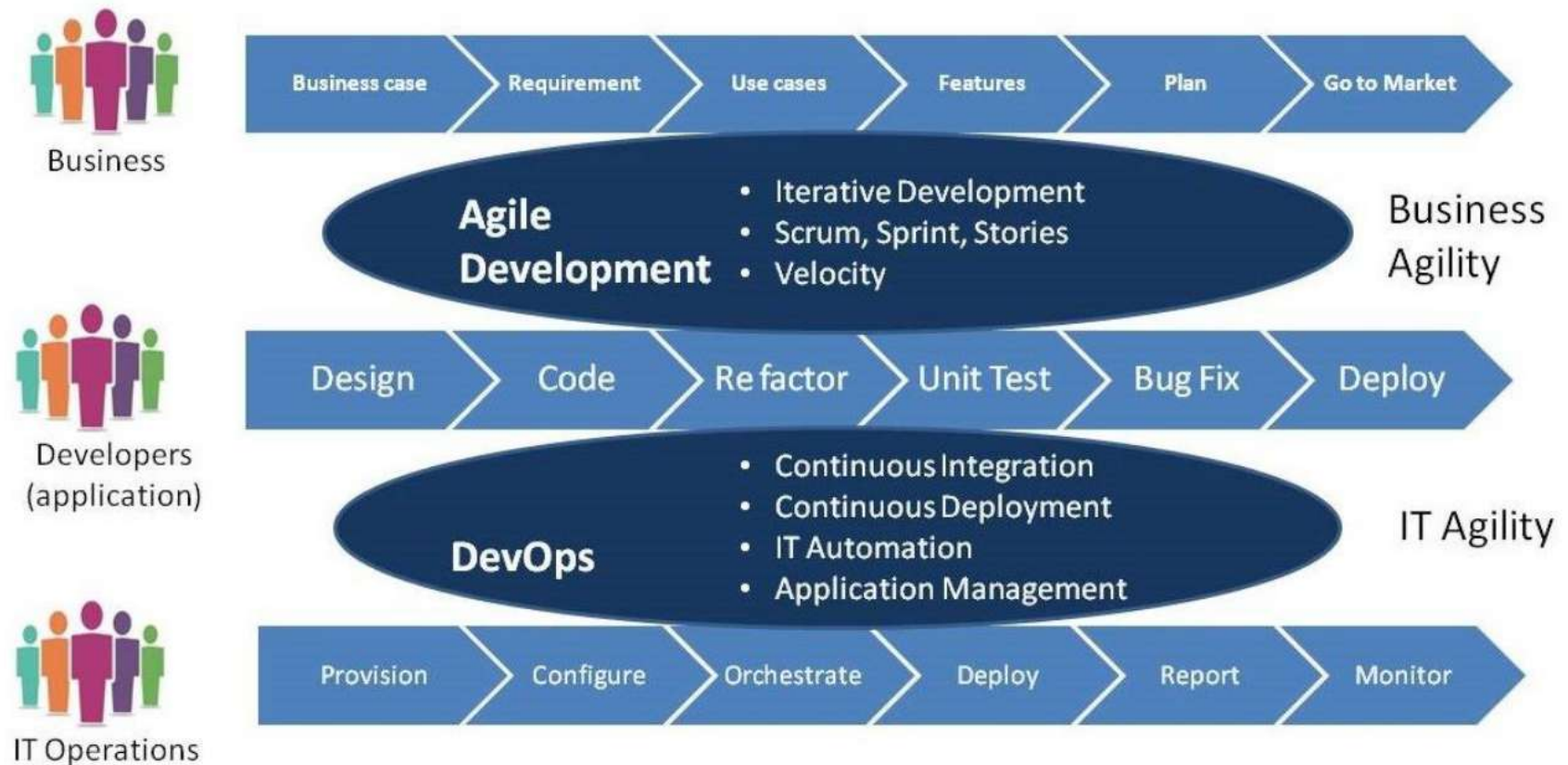
I “muri” dello sviluppo IT



Il “muro” tra Dev e Ops



Superare i “muri”



Un Team Dev efficace non si limita a “consegnare codice”: si interessa a deploy, osservabilità, rollback e qualità in esercizio.

DevOps è parte di Agile

- ◆ DevOps è guidata da diversi fattori:
 - Utilizzo delle metodologie agili e lean di sviluppo del software per: feedback rapido, riduzione degli sprechi, miglioramento continuo.
 - Necessità di incrementare la frequenza dei rilasci in produzione
 - Ampia disponibilità di un'infrastruttura virtualizzata e in cloud
 - Uso di strumenti di configuration management e Infrastructure as Code



DevOps si basa su:

- ◆ Virtualizzazione dei datacenter (o cloud)
- ◆ Strumenti di automazione
- ◆ Metodologie di lavoro uniformi e condivise fra sviluppatori e sistemisti (di tipo agile/lean)
- ◆ Approccio incrementale-iterativo sino al rilascio in produzione, con iterazioni brevi
- ◆ Testing continuo e automatizzato, e feedback continuo tra le varie fasi

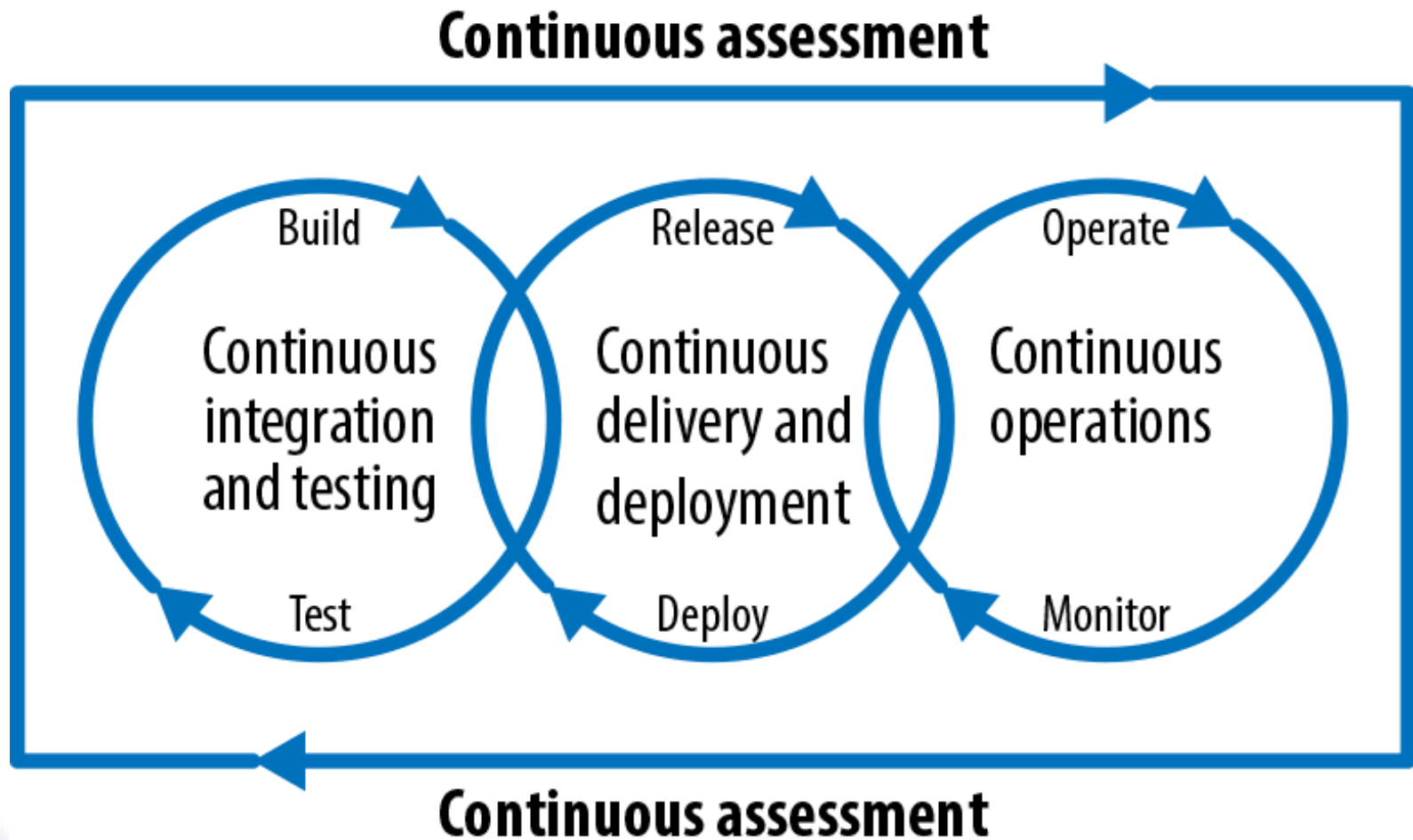


CALMS

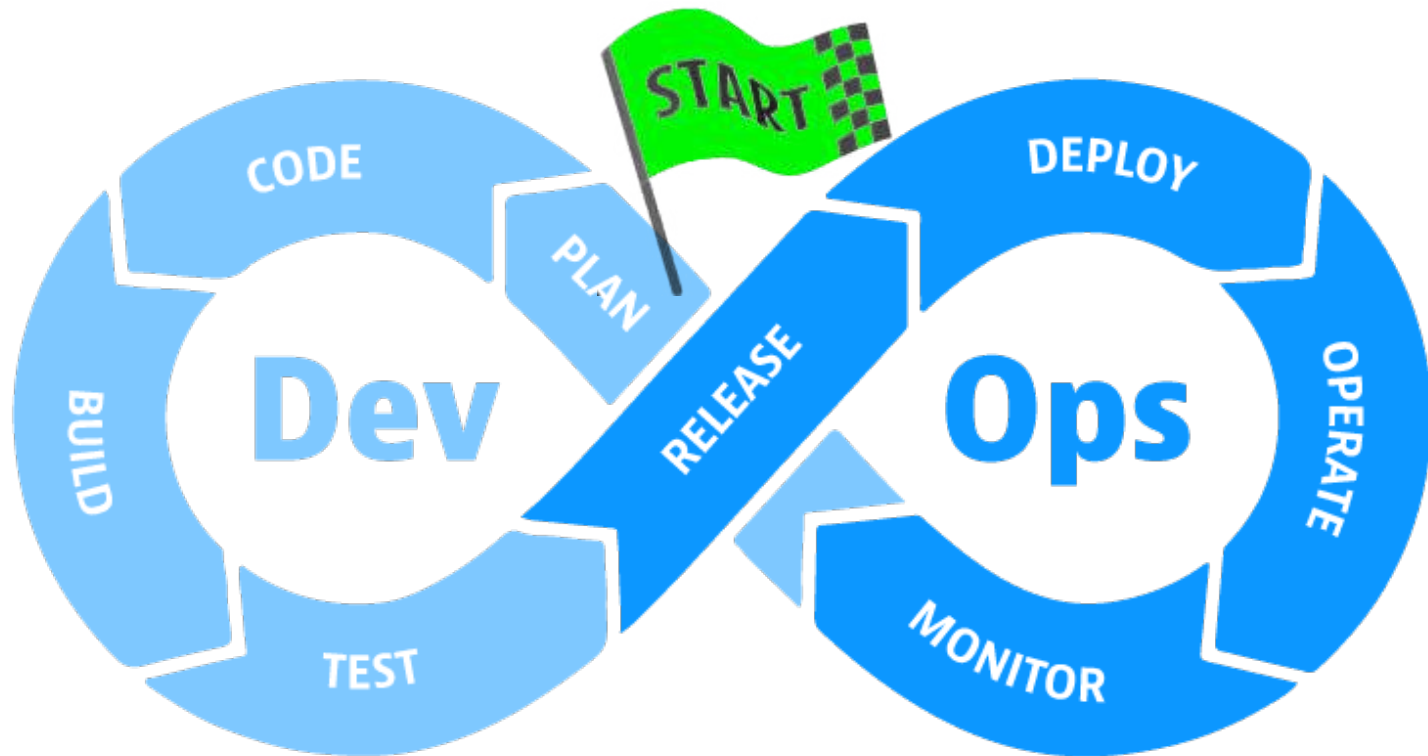
- ◆ I principi DevOps sono sintetizzati nell'acronimo C.A.L.M.S.:
 - **Culture**: gestire il cambiamento focalizzandosi sulla collaborazione
 - **Automation**: eliminare le azioni manuali ripetitive (Continuous Integration, Continuous Delivery, Infrastructure-as-a-code)
 - **Lean**: utilizzare i principi Lean per velocizzare, standardizzare e rendere efficienti le attività
 - **Metrics**: misurare qualsiasi cosa, utilizzando i risultati per rifinire costantemente le attività
 - **Sharing**: condividere le esperienze di successo e di fallimento per una crescita diffusa



I cicli automatizzati di DevOps



Gli stadi di DevOps



- Ogni stadio può avere strumenti diversi, ma il principio è mantenere continuità, tra feedback lungo l'intero ciclo.

Plan

- ◆ Le attività di pianificazione di un sistema utilizzano il *business value* e la specifica dei requisiti.
- ◆ Esse comprendono:
 - Valutazione e aggiornamento del *business plan*
 - Requisiti (backlog, USs, requisiti non funzionali...)
 - Metriche legate al *business*
 - Aggiornamento delle metriche del rilascio
 - Piano di rilascio, tempistica e business case
 - Politica e requisiti di sicurezza



Code e Build

- ◆ Progetto, codifica e configurazione del software
- ◆ Le attività specifiche sono:
 - Progettazione del software e gestione della configurazione
 - Codifica, incluso versionamento, controllo di qualità e delle prestazioni
 - Build del software: produzione di codice binario, immagini container, pacchetti o versioni rilasciabili, incluso controllo di prestazioni del software
 - Un buon flusso di build deve essere veloce, affidabile e il più possibile deterministico



Test

- ◆ La verifica è direttamente associata alla garanzia della qualità del software rilasciato
- ◆ Le principali attività in questo ambito sono:
 - Test di accettazione
 - Test di regressione
 - Analisi della sicurezza e delle vulnerabilità
 - Misura delle prestazioni
 - Test della configurazione
- ◆ Le attività della verifica sono di solito basate su: test automatici, analisi statica, analisi di sicurezza



Package

- ◆ Il Package si riferisce alle attività coinvolte una volta che la versione è pronta per la distribuzione, spesso definita anche *staging* o preproduzione
- ◆ Qui diventano centrali naming, versionamento, dipendenze esplicite e immutabilità dei prodotti
- ◆ Ciò include compiti e attività come:
 - Approvazione/pre-approvazione
 - Configurazione del package
 - *Triggered releases*
 - *Release staging and holding*



Release

- ◆ Le attività correlate al rilascio del software verso l'ambiente di destinazione includono:
 - pianificazione
 - orchestrazione
 - provisioning (messa a disposizione delle risorse esterne necessarie, come rete, DB, ecc.)
 - distribuzione del software
- ◆ Con per target l'ambiente di destinazione finale
- ◆ Con batch piccoli e flusso automatizzato, il rischio del rilascio diminuisce e la frequenza può aumentare.



Configure

- ◆ L'attività di configurazione riguarda configurazione di ambienti, provisioning, secret management, rete e dipendenze infrastrutturali.
- ◆ L'idea chiave di Infrastructure as Code è che anche l'infrastruttura debba essere tracciabile, versionata e riproducibile
- ◆ I principali tipi di soluzioni che facilitano queste attività sono l'automazione continua della configurazione, la gestione della configurazione e gli strumenti di infrastruttura programmati come codice
- ◆ Ciò riduce differenze tra ambienti e il classico problema: *“sul mio computer funziona”*.



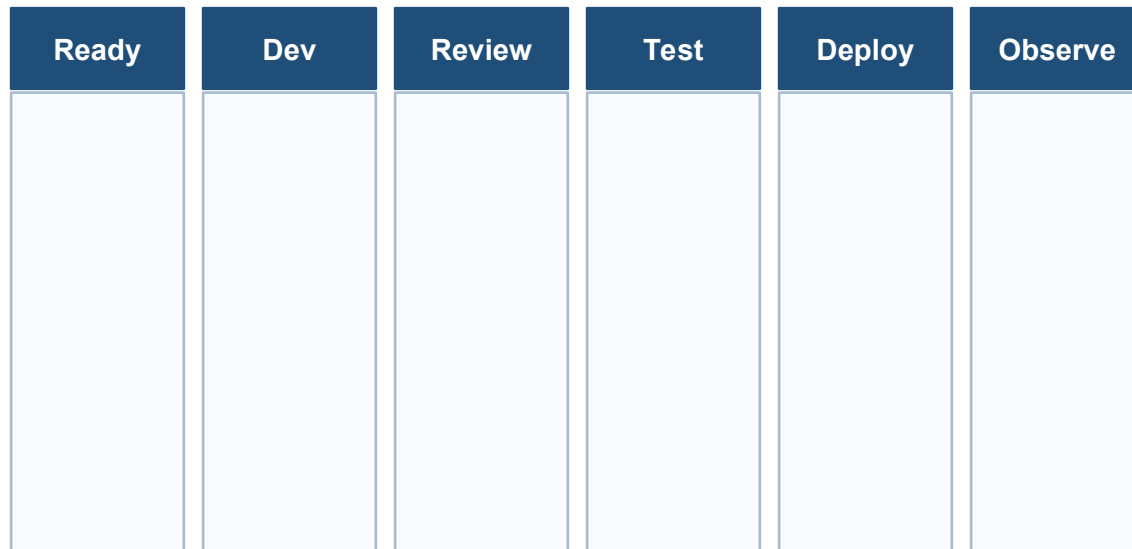
Monitor

- ◆ Il monitoraggio è una fase importante e permanente di DevOps
- ◆ Esso consente all'organizzazione IT di identificare problemi di versioni specifiche e di comprenderne l'impatto sugli utenti finali.
- ◆ Esso comprende la misura e il controllo di:
 - Prestazioni dell'infrastruttura IT
 - Risposta ed esperienza dell'utente finale
 - Metriche e statistiche di produzione
- ◆ Le informazioni del monitoraggio influiscono sulle attività di Plan del ciclo successivo



Kanban e DevOps

- ◆ Per chi usa Kanban, il punto cruciale è estendere il tabellone fino al deploy e al feedback operativo
- ◆ Se il flusso termina a “Done”, si perde visibilità su rilasci, attese di approvazione, deploy...
- ◆ In DevOps il tabellone Kanban rende visibili anche i colli di bottiglia tecnici e operativi



Metriche di flusso e metriche DevOps

Metrica	Significato
Lead time	Tempo totale da richiesta a disponibilità in produzione.
Cycle time	Tempo tra inizio effettivo e completamento del lavoro.
Throughput	Numero di elementi completati nell'unità di tempo.
Change failure rate	Percentuale di rilasci che causano incidenti o rollback.
MTTR	Tempo medio di ripristino dopo un problema.



Esempio di DevOps

- ◆ **Scenario:** vogliamo sviluppare una piccola web app con Node.js e React.
- ◆ **Obiettivo:** Vogliamo automatizzare il processo di sviluppo, test, rilascio e monitoraggio.



Esempio di DevOps

◆ **Stack usato:**

- GitHub – versionamento del codice
- GitHub Actions – CI/CD automation
- Docker – containerizzazione
- Heroku / Render / Vercel – deploy automatico
- Prometheus + Grafana – monitoraggio base (facoltativo per semplicità)
- Slack – notifiche



Esempio di DevOps

♦ **Flusso semplificato:**

- 1) Dev scrive il codice in un branch (feature/login) e fa il push su GitHub
- 2) CI: Integrazione Continua (GitHub Actions): si attiva una pipeline automatica che:
 - 1) Installa dipendenze (npm install)
 - 2) Esegue i test (npm test)
 - 3) Costruisce l'app (npm run build)
 - 4) Crea un'immagine Docker (docker build)
- 3) CD: Deploy Continuo: se i test passano il deploy dell'app è fatto automaticamente sul provider (Heroku), in alternativa, si effettua un “push” su Docker Hub dell'immagine Docker .



Esempio di DevOps

- 4) Monitoraggio: Un semplice health-check API viene controllato da uno script o da UptimeRobot. Se c'è un errore o downtime, viene inviata una notifica su Slack.
- 5) Feedback: Se ci sono errori in test o nel deploy, GitHub Actions fallisce e notifica il team via Slack o email. Il dev corregge, fa push, e tutto riparte.



Esempio di DevOps

- ◆ Risultato:
 - Ogni push viene testato.
 - Ogni merge su main porta a un deploy automatico.
 - Nessuna azione manuale per test o deploy.
 - Feedback immediato per i bug.

